

Greedy Algorithms

A Complete Learning Guide

"Don't ask only: What looks best right now?"

*"Ask: Why does this choice leave me in at least as good a position
for the future as every other choice?"*

15 Problems · 8 Proof Patterns · Greedy vs DP · Full Code

Table of Contents

Part 1 — General Greedy Thinking Framework

Part 2 — How to Recognize a Greedy Problem

Part 3 — The Exchange / Swap Test

Part 4 — Problems 1–15 (All Solved Problems)

Part 5 — Visualization Reference

Part 6 — Greedy Proof Patterns (8 Patterns)

Part 7 — Greedy vs Dynamic Programming

Part 8 — Counterexample Training

Part 9 — Greedy Decision Tree

Part 10 — Problem-Solving Template

Part 1 — General Greedy Thinking Framework

When you encounter an optimization or feasibility problem, run through these 14 steps in order. Each question builds on the previous one. The framework is not a memorization exercise — the questions flow naturally from each other.

DISCOVER

- 1 **What am I choosing?** Name the repeated decision physically
- 2 **What am I optimizing?** Maximize / minimize / feasibility
- 3 **What is the constraint?** What makes a choice invalid?
- 4 **What resource is scarce?** Time? Capacity? Flexibility?
- 5 **What future damage?** What does a bad choice destroy?
- 6 **Candidate rule** "At every step, choose the ___ option"

ATTACK

- 7 **Counterexample** Try small, extreme, equal, outlier inputs
- 8 **Subproblem repeats?** Same structure after one choice?
- 9 **Future dependence?** If yes → likely DP, not greedy

PROVE

- 10 **Proof type** Exchange / dominance / forced / range
- 11 **Swap test** Replace OPT's choice with G → still valid?

IMPLEMENT

- 12 **State compression** Minimal state to continue?
- 13 **Ordering structure** Sort / heap / two-ptr / hash?
- 14 **The Master Flow Diagram** → state → feasibility → take → update



Why This Order Matters

Steps 1–6 (Discover) establish what you are doing and propose a candidate. Steps 7–9 (Attack) try to break the candidate before you invest time proving it — a 30-second counterexample saves 20 minutes. Steps 10–11 (Prove) give the formal justification. Steps 12–14 (Implement) translate the proven rule into code.

Key Insight

The most important question in the entire framework is Step 5: "What future damage can a bad choice cause?" The greedy rule almost always emerges directly from the answer.

Step 4 — Scarce Resources by Problem Type

Scarce Resource

Problems

Time	Activity selection, scheduling, stock trading
Weight / capacity	Knapsack variants, fractional problems
Flexibility	Lemonade change (\$5 more flexible than \$10)
Connectivity	Minimum spanning tree (Kruskal, Prim)
Deadline slots	Job scheduling with deadlines
Split opportunities	Partition labels
Card counts	Hand of Straights

Step 9 — Greedy vs DP Quick Test

After proposing a rule, ask: "Does my current choice depend on decisions I haven't made yet?" If no → greedy candidate. If yes → likely DP.

Signal	Example
Past collapses to 1–2 variables	Activity selection: just lastEnd
Past branches into many states	0/1 Knapsack: (index, capacity)
Choice is purely local (today vs tomorrow)	Stock II: just consecutive days
Future prices affect sell decision	Stock with fee: must consider future

Part 2 — How to Recognize a Greedy Problem

Positive Signals

- Local choices feel natural — making the "best" decision now feels intuitively correct
- Sorting exposes the order — many greedy problems become easy once sorted by the right key
- Smallest/largest feasible item wins — "take the smallest that works" or "take the largest" is provably safe
- Flexible resources should be preserved — use less flexible resources first
- Forced choices — sometimes there is only one logically valid option (smallest card must start a group)

Sorting Keys by Problem Type

Sort Key	Problem
Sort by finish time	Activity selection, Burst balloons
Sort by greed factor	Assign cookies
Sort by value	Array pair sum
Sort by end position	Hand of Straights
Precompute last occurrence	Partition labels
Sort descending	Largest perimeter triangle

Warning — "Greedy-Looking" is NOT Enough

A rule that works on several examples is not proven correct. You need either a counterexample (to reject) or an exchange argument (to accept).

Common Trap

Highest value item first (0/1 Knapsack) — combining smaller items can give higher total value.

Common Trap

Largest coin first (arbitrary denominations) — e.g. coins [1,3,4], target 6: greedy gives $4+1+1=3$ coins, optimal is $3+3=2$ coins.

Common Trap

Earliest starting activity — does not account for how long the activity runs; an early-starting long activity blocks many future activities.

Part 3 — The Exchange / Swap Test

The exchange argument is the standard method for proving that a greedy choice is safe. It answers: "Even if the optimal solution makes a different choice, can I show my greedy choice is just as good?"

Exchange / Swap Argument — General Pattern

OPT: **X** → Y → Z → ... (optimal sequence)

↓ replace X with G (greedy choice)

OPT': **G** → Y → Z → ... (still valid, still optimal)

✓ Greedy choice G is safe — an optimal solution starting with G always exists

The General Proof Template

Generic Exchange Proof Template

Let G = your greedy choice

Let OPT = any optimal solution

Let X = OPT's first choice (different from G)

Case 1: OPT already starts with G. Done.

Case 2: OPT starts with $X \neq G$.

- Replace X with G in OPT.
- Show all subsequent choices remain valid.
(Usually: G is "at least as good" as X in the relevant dimension.)
- Show the objective value does not decrease.
- Call this OPT'. OPT' is valid and at least as good as OPT.
- OPT' starts with G.

Conclusion: There always exists an optimal solution starting with G.

By induction: the full greedy sequence is optimal.

Four Levels of Justification — Do Not Confuse Them

Intuition	"It feels like choosing X is safe" — Weak, often wrong
Example testing	"It works on 5 examples" — Weak, counterexamples might exist
Exchange argument	"I can always swap my choice into OPT without loss" — Strong, this is a proof
Formal proof	Induction or structural proof — Strongest, but often unnecessary

Key Insight

You do not need to check every possible pair of solutions. You just need to show that any optimal solution can be transformed to use your greedy choice without loss.

Part 4 — Greedy Problems We Solved

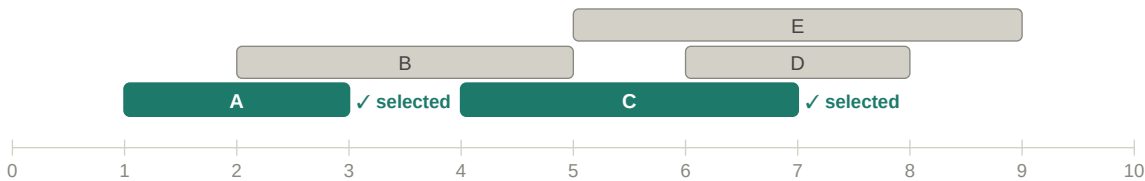
Problem 1 Activity Selection

Medium

Given n activities with start/finish times, select the maximum number of non-overlapping activities.

Activity Selection — Timeline

Selected (greedy: earliest finish) Skipped



Greedy Derivation

Choosing	Which activity to attend next
Optimizing	Maximize number of activities attended
Constraint	Activities cannot overlap
Scarce resource	Time — passes irreversibly
Future damage	Long activities destroy more future time
Candidate rule	Choose the activity with the earliest finish time

Exchange / Swap Test

Exchange Argument

Let G = earliest finishing activity ($\text{finish}(G) \leq \text{finish}(X)$)
Let X = OPT's first choice

Every activity after X has $\text{start} \geq \text{finish}(X)$.
Since $\text{finish}(G) \leq \text{finish}(X)$, those activities also have $\text{start} \geq \text{finish}(G)$.
Replacing X with G : same activities remain compatible.
Same count $\rightarrow$ objective unchanged. ✓

Code

```

1 def activity_selection(activities):
2     activities.sort(key=lambda x: x[1]) # sort by finish time
3     last_end = 0
4     count = 0
5     for start, end in activities:
6         if start >= last_end:
7             count += 1
8             last_end = end
9     return count

```

python

Time: $O(n \log n)$ — sorting. Space: $O(1)$.

Common Trap

Sorting by start time or duration instead of finish time. An early-starting long activity blocks many future activities.

Problem 2

Longest Palindrome

Easy

Given a string, return the length of the longest palindrome buildable from its characters. Case-sensitive.

Choosing	How many copies of each character to include
Optimizing	Maximize total palindrome length
Constraint	Characters outside center must appear in pairs
Candidate rule	Take the largest even count per character + one center

Longest Palindrome — Visual

"abcccccdd" → a=1, b=1, c=4, d=2

c: ■■■■ → take 4 (2 left, 2 right)

d: ■■ → take 2 (1 left, 1 right)

a: ■ → leftover → use as center

Palindrome: d c [a] c d → "dccaccd" (length 7)

```

1 def longest_palindrome(s):
2     count = {}
3     for ch in s:
4         count[ch] = 1 + count.get(ch, 0)
5     answer = 0
6     has_odd = False
7     for freq in count.values():
8         answer += freq - (freq % 2) # largest even portion
9         if freq % 2 == 1:
10             has_odd = True
11     return answer + (1 if has_odd else 0)

```

python

Time: $O(n)$. Space: $O(k)$ where k = distinct characters.

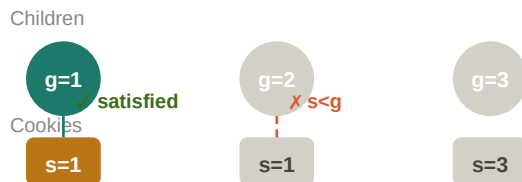
Common Trap

Forgetting the center character. Also: adding more than one center character — only one center slot exists.

Problem 3**Assign Cookies**

Easy

Each child has a greed factor $g[i]$. Each cookie has size $s[j]$. Cookie j satisfies child i if $s[j] \geq g[i]$. Maximize content children.

Assign Cookies — Greedy Pairing

Choosing	Which cookie to give which child
Scarce resource	Large cookies that satisfy demanding children
Future damage	Wasting a large cookie on an easy child
Candidate rule	Give smallest sufficient cookie to least demanding child

```

1 def assign_cookies(g, s):
2     g.sort()
3     s.sort()
4     child = 0
5     for cookie in s:
6         if child < len(g) and cookie >= g[child]:
7             child += 1
8     return child

```

python

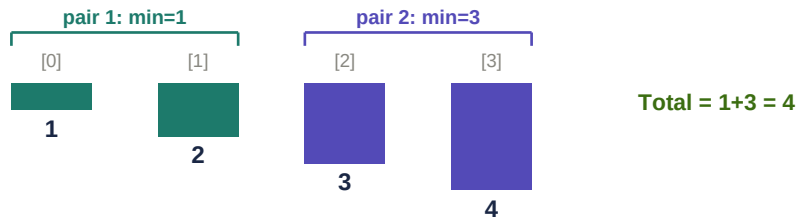
Time: $O(n \log n + m \log m)$. Space: $O(1)$.

Problem 4**Array Pair Sum**

Easy

Group $2n$ integers into n pairs. Score = sum of pair minimums. Maximize score.

Array Pair Sum — Adjacent Pairing (sorted)



Exchange argument: for any sorted $a \leq b \leq c \leq d$, adjacent pairing gives $a+c$ while non-adjacent gives $a+b \leq a+c$. Adjacent always wins.

```
1 def array_pair_sum(nums):
2     nums.sort()
3     return sum(nums[i] for i in range(0, len(nums), 2))
```

python

Time: $O(n \log n)$. Space: $O(1)$.

Problem 5

Lemonade Change

Easy

Customers pay \$5/\$10/\$20 for \$5 lemonade. Give exact change. Return true if possible.

Lemonade Change — Resource Flexibility



Scarce resource	\$5 bills — needed for both \$10 and \$20 customers
Candidate rule	For \$20: prefer \$10+\$5 over \$5+\$5+\$5
Why	\$5 is more flexible — preserve it for \$10 customers

```
1 def lemonade_change(bills):
2     five = ten = 0
3     for bill in bills:
4         if bill == 5:
5             five += 1
6         elif bill == 10:
7             if five == 0: return False
8             five -= 1; ten += 1
9         else: # 20
10            if ten > 0 and five > 0: ten -= 1; five -= 1
11            elif five >= 3: five -= 3
12            else: return False
13     return True
```

python

Time: $O(n)$. Space: $O(1)$.

Problem 6

DI String Match

Easy

Given string of I/D, construct permutation of $[0..n]$ where I means increase, D means decrease.

DI String Match — Two Pointer Trace

`s = "IDID", lo=0, hi=4`

I → place lo=0 → [0, ...] (lo becomes 1)
 D → place hi=4 → [0, 4, ...] (hi becomes 3)
 I → place lo=1 → [0, 4, 1, ...] (lo becomes 2)
 D → place hi=3 → [0, 4, 1, 3, ...] (hi becomes 2)
 Last: lo=hi=2 → [0, 4, 1, 3, 2]

Check: $0 < 4$ ✓ $4 > 1$ ✓ $1 < 3$ ✓ $3 > 2$ ✓

```
1 def di_string_match(s):
2     lo, hi = 0, len(s)
3     result = []
4     for ch in s:
5         if ch == "I":
6             result.append(lo); lo += 1
7         else:
8             result.append(hi); hi -= 1
9     result.append(lo) # lo == hi at this point
10    return result
```

python

Time: $O(n)$. Space: $O(n)$ for output.

Problem 7

Minimum Arrows to Burst Balloons

Medium

Balloons as intervals $[xstart, xend]$. Arrows shot vertically burst all balloons whose range contains the arrow x-position. Minimize arrows.

Burst Balloons — Timeline

Sorted by end: `[[1,6], [2,8], [7,12], [10,16]]`

Timeline:

```
[1■■■■■■■■■■6]
[2■■■■■■■■■■8]
      [7■■■■■■■■■■12]
            [10■■■■■■■■■■16]
```

Arrow at $x=6$: bursts $[1,6]$ and $[2,8]$ ← shoot at earliest end

Arrow at $x=12$: bursts $[7,12]$ and $[10,16]$

Answer: 2 arrows

Key Insight

This is activity selection in disguise. Instead of maximizing activities, we minimize arrows by maximizing what each arrow covers. Same sort key: earliest end.

```

1 def find_min_arrow_shots(points):
2     if not points: return 0
3     points.sort(key=lambda x: x[1]) # sort by end
4     arrows = 1
5     end = points[0][1]
6     for start, finish in points[1:]:
7         if start > end: # gap — need new arrow
8             arrows += 1
9             end = finish
10    return arrows

```

python

Time: $O(n \log n)$. Space: $O(1)$.

Problem 8**Best Time to Buy and Sell Stock II**

Medium

Given daily prices, buy and sell any number of times. Maximize profit.

Stock II — Collect Every Upward Step

prices = [7, 1, 5, 3, 6, 4]

```

  7
    5      6
      3    4
    1

```

Day 1→2: 1→5 ▲ profit += 4
 Day 2→3: 5→3 ▼ skip
 Day 3→4: 3→6 ▲ profit += 3
 Day 4→5: 6→4 ▼ skip

Total: 7 (same as buying at 1 selling at 5, buying at 3 selling at 6)

Key Insight

Key identity: $(5-1) = (2-1) + (3-2) + (5-3)$. Collecting every consecutive upward step is mathematically equivalent to any peak-to-valley capture. Intermediate terms cancel.

```

1 def max_profit(prices):
2     profit = 0
3     for i in range(len(prices) - 1):
4         if prices[i+1] > prices[i]:
5             profit += prices[i+1] - prices[i]
6     return profit

```

python

Time: $O(n)$. Space: $O(1)$.

Common Trap

Stock with Transaction Fee: adding a fee creates coupling between decisions. Selling today affects whether the next trade is worth it — this becomes DP, not greedy.

Problem 9**Hand of Straights**

Medium

Given cards and groupSize k, determine if cards can be rearranged into groups of k consecutive cards.

Choosing	Which card to start each new group with
Constraint	Groups must contain k consecutive cards
Scarce resource	Count of each card value
Forced rule	Smallest available card MUST start a group
Why forced	It has no predecessor — can only be a group start

```

1 from collections import Counter
2
3 def is_n_straight_hand(hand, group_size):
4     if len(hand) % group_size != 0: return False
5     count = Counter(hand)
6     for val in sorted(count):
7         if count[val] > 0:
8             groups = count[val] # must form this many groups here
9             for i in range(group_size):
10                if count.get(val + i, 0) < groups: return False
11                count[val + i] -= groups
12     return True

```

python

Time: $O(n \log n)$. Space: $O(n)$.

Common Trap

Forming one group at a time when a value appears multiple times. If value v appears k times, form k groups simultaneously starting at v.

Problem 10**Merge Triplets to Form Target**

Medium

Triplets can be merged using element-wise max. Determine if target is reachable.

Key Insight

Merging can only INCREASE values. Any triplet exceeding the target in any dimension must be excluded — it would irreversibly damage the result. This makes the choice forced, not a guess.

```

1 def merge_triplets(triplets, target):
2     x, y, z = target
3     found_x = found_y = found_z = False
4     for a, b, c in triplets:
5         if a > x or b > y or c > z: continue
6         if a == x: found_x = True
7         if b == y: found_y = True
8         if c == z: found_z = True
9     return found_x and found_y and found_z

```

python

Time: $O(n)$. Space: $O(1)$.

Problem 11

Partition Labels

Medium

Split string into max substrings where each letter appears in at most one substring.

Partition Labels — Trace

s = "xyxxyzbbbisl"

Last occurrences: x→3, y→4, z→7, b→8, i→10, s→11, l→12

```

i=0(x): furthest=3
i=1(y): furthest=4
i=2(x): furthest=4
i=3(x): furthest=4
i=4(y): furthest=4 → i==furthest → SPLIT! size=5

```

```

i=5(z): furthest=7 ... i=8(b): furthest=8 → SPLIT! size=5
i=10(i)→SPLIT size=1, i=11(s)→SPLIT size=1, i=12(l)→SPLIT size=1

```

Result: [5, 5, 1, 1, 1]

```

1 def partition_labels(s):
2     last = {ch: i for i, ch in enumerate(s)}
3     result = []
4     start = furthest = 0
5     for i, ch in enumerate(s):
6         furthest = max(furthest, last[ch])
7         if i == furthest:
8             result.append(i - start + 1)
9             start = i + 1
10    return result

```

python

Time: $O(n)$. Space: $O(1)$ — at most 26 characters.

Problem 12

Valid Palindrome II

Easy

Return true if string can become a palindrome after deleting at most one character.

Valid Palindrome II — Two Pointer

```
s = "abca"
```

```
Two pointer:  a b c a
               ↑   ↑
               L   R
```

```
a == a → move inward
b != c → mismatch!
```

```
Try delete b: check "aca" (indices L+1..R) → palindrome ✓
```

```
Try delete c: check "aba" (indices L..R-1) → palindrome ✓
```

```
Either works → return True
```

```
1 def valid_palindrome(s):
2     def is_palindrome(left, right):
3         while left < right:
4             if s[left] != s[right]: return False
5             left += 1; right -= 1
6         return True
7     L, R = 0, len(s) - 1
8     while L < R:
9         if s[L] != s[R]:
10            return (is_palindrome(L+1, R) or
11                    is_palindrome(L, R-1))
12        L += 1; R -= 1
13    return True
```

python

Time: $O(n)$. Space: $O(1)$.

Problem 13

Valid Parenthesis String

Medium

String with (,), * (can be (,), or empty). Return true if valid.

Key Insight

Instead of committing * to a value early, track the RANGE [low, high] of possible unmatched open-bracket counts. Defer the decision — only check if 0 is reachable at the end.

Character	Update
(encountered	low += 1, high += 1
) encountered	low -= 1, high -= 1
* encountered	low -= 1, high += 1 (min assumes), max assumes ()
high < 0	Too many) even in best case → return False

Clamp	<code>low = max(low, 0)</code> (can always use <code>*</code> as empty)
End	<code>low == 0</code> → True

```

1 def check_valid_string(s):
2     low = high = 0
3     for ch in s:
4         if ch == "(":
5             low += 1; high += 1
6         elif ch == ")":
7             low -= 1; high -= 1
8         else: # *
9             low -= 1; high += 1
10        if high < 0: return False
11        low = max(low, 0)
12    return low == 0

```

Time: $O(n)$. Space: $O(1)$.

Problem 14

Largest Perimeter Triangle

Easy

Given array of lengths, find the largest perimeter of a triangle with non-zero area. Triangle condition: $a + b > c$ for sorted $a \leq b \leq c$.

```

1 def largest_perimeter(nums):
2     nums.sort(reverse=True)
3     for i in range(len(nums) - 2):
4         if nums[i] < nums[i+1] + nums[i+2]:
5             return nums[i] + nums[i+1] + nums[i+2]
6     return 0

```

Sort descending. Check consecutive triples — first valid triple gives the largest perimeter. Time: $O(n \log n)$. Space: $O(1)$.

Problem 15

Candy

Hard

Give candies to children in a line. Each gets ≥ 1 . Higher rating than a neighbor → more candies than that neighbor. Minimize total.

Candy — Two-Pass Greedy

```
ratings = [1, 0, 2]
```

Left pass ($\rightarrow$): ensure each > left neighbor if rating higher

```
left = [1, 1, 2] (rating[2]=2 > rating[1]=0  $\rightarrow$  left[2]=left[1]+1)
```

Right pass ($\leftarrow$): ensure each > right neighbor if rating higher

```
right = [2, 1, 1] (rating[0]=1 > rating[1]=0  $\rightarrow$  right[0]=right[1]+1)
```

Result = $\max(\text{left}[i], \text{right}[i]) = [2, 1, 2] \rightarrow \text{sum} = 5$

Why max, not left+right?

```
max(3,3) = 3 (one value satisfies both constraints)
```

```
3+3-1 = 5 (overcounts when both > 1)
```

Key Insight

Two passes satisfy each direction independently. $\max(\text{left}[i], \text{right}[i])$ is the minimum single value satisfying both constraints simultaneously — any less violates one side.

```
1 def candy(ratings):
2     n = len(ratings)
3     left = [1] * n
4     right = [1] * n
5     for i in range(1, n):
6         if ratings[i] > ratings[i-1]:
7             left[i] = left[i-1] + 1
8     for i in range(n-2, -1, -1):
9         if ratings[i] > ratings[i+1]:
10            right[i] = right[i+1] + 1
11    return sum(max(left[i], right[i]) for i in range(n))
```

python

Time: $O(n)$. Space: $O(n)$.

Common Trap

Using left+right instead of max. If $\text{left}[i]=3$ and $\text{right}[i]=3$, the child needs 3 candies total, not 5.

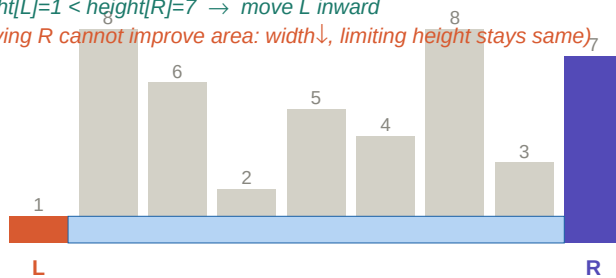
Part 5 — Visualization Reference

Container With Most Water — Two Pointer

Container With Most Water — Move Shorter Pointer

$height[L]=1 < height[R]=7 \rightarrow \text{move } L \text{ inward}$

(moving R cannot improve area: width $\downarrow$, limiting height stays same)



Exchange Argument — General Pattern

Exchange / Swap Argument — General Pattern

OPT: **X** $\rightarrow Y \rightarrow Z \rightarrow \dots$ (optimal sequence)

$\downarrow$ replace X with G (greedy choice)

OPT': **G** $\rightarrow Y \rightarrow Z \rightarrow \dots$ (still valid, still optimal)

✓ Greedy choice G is safe — an optimal solution starting with G always exists

Lemonade Change — Resource Flexibility

Lemonade Change — Resource Flexibility



Part 6 — Greedy Proof Patterns

After solving many greedy problems, the same underlying arguments appear repeatedly. Recognizing the pattern saves you from re-inventing the proof.

Pattern A — Preserve Future Capacity

Idea: Choice consuming least of the scarce resource is always safe.

Proof: Exchange argument: G consumes less than X, so all choices after X also work after G.

Examples: Activity selection (time), Burst balloons (x-axis range)

Pattern B — Smallest Sufficient Resource

Idea: Use the minimum resource that satisfies current demand; preserve larger resources.

Proof: Exchange: swapping large for small in OPT never makes remaining demands impossible.

Examples: Assign cookies, Lemonade change (\$5 for \$10 customer)

Pattern C — Preserve Flexible Resources

Idea: When two resources satisfy the same demand, use the less flexible one.

Proof: Dominance: flexible resource is always at least as useful → preserve it.

Examples: Lemonade change (\$10 before \$5 for \$20), DI string match

Pattern D — Adjacent Pairing

Idea: Pair adjacent sorted elements to prevent large numbers being wasted.

Proof: Exchange with 4 elements: non-adjacent gives $a+b \leq a+c = \text{adjacent}$.

Examples: Array pair sum

Pattern E — Eliminate Dominated Choices

Idea: If one direction can be proven to never improve, always move the other.

Proof: Dominance: moving the taller pointer keeps the same height limit with reduced width.

Examples: Container with most water

Pattern F — Sort First, Then Scan

Idea: Sorting brings the best candidate to front at each step, making choice automatic.

Proof: N/A — sorting is the implementation of the greedy order, not the proof.

Examples: Activity selection, Cookies, Pair sum, Balloons, Hand of Straights

Pattern G — Forced Choices

Idea: Sometimes the greedy rule is the only logically valid option.

Proof: No exchange argument needed — the alternative is simply invalid.

Examples: Hand of Straights (smallest must start group), Merge Triplets (exceed → skip)

Pattern H — Range Tracking

Idea: Track range of all possible values instead of committing to one early.

Proof: Defer decision until forced; check if target value (0) is reachable.

Examples: Valid Parenthesis String

Part 7 — Greedy vs Dynamic Programming

GREEDY

- One choice proven permanently safe
- Commit — never reconsider
- Past → tiny state (lastEnd)
- Future does NOT affect choice
- $O(n \log n)$ typical

DYNAMIC PROGRAMMING

- No choice is always safe
- Preserve multiple possibilities
- Past branches into many states
- Future DOES affect current choice
- $O(n^2)$ or $O(n \cdot k)$ typical

Key question: "Does my current choice depend on decisions I haven't made yet?"

The State Compression Test

After one greedy choice, what do you need to remember?

State after one choice	Verdict
Activity selection: just lastEnd	GREEDY ✓
Lemonade change: just five, ten	GREEDY ✓
0/1 Knapsack: (index, capacity) — branches	DP
Coin change: remaining amount — many states	DP
Weighted intervals: which chosen affects future compatibility	DP

Classic Greedy Failures

Fail 1 — 0/1 Knapsack

Fail 1 — 0/1 Knapsack

Tempting rule: take items in order of highest value/weight ratio.

Counterexample:

Capacity = 50

Item A: weight=10, value=60 (ratio=6.0) ← greedy picks first

Item B: weight=20, value=100 (ratio=5.0) ← greedy picks second

Item C: weight=30, value=120 (ratio=4.0) ← can't fit (10+20+30=60>50)

Greedy: A+B → weight=30, value=160

Optimal: B+C → weight=50, value=220

Why: Choosing A uses capacity that could fit part of a better combination.

Future value depends on which items were already taken → DP.

Fail 2 — Coin Change (Arbitrary Denominations)

Fail 2 — Coin Change

Tempting rule: always use the largest coin $\leq$ remaining amount.

Counterexample:

Coins = [1, 3, 4], Target = 6

Greedy: $4 + 1 + 1 = 3$ coins

Optimal: $3 + 3 = 2$ coins

Why: Choosing 4 leaves remainder 2, which needs two 1s.

Choosing 3 leaves remainder 3, which needs one 3.

Optimal coins for amount 6 depends on optimal for 2 and 3 $\rightarrow$ DP.

Stock II vs Stock with Transaction Fee

Stock II vs Stock with Fee

Stock II (no fee):

$(5-1) = (2-1) + (3-2) + (5-3)$ – consecutive steps always collectable

Decision at day i only depends on $\text{prices}[i]$ and $\text{prices}[i+1]$ $\rightarrow$ GREEDY

Stock with Fee:

Selling at 3 and rebuying at 2 costs an extra fee.

Sometimes holding through a small dip is better than selling and rebuying.

Decision today depends on whether future price will justify a new trade $\rightarrow$ DP

DP states:

cash = max profit when NOT holding

hold = max profit when holding

Transitions:

cash = $\max(\text{cash}, \text{hold} + \text{price} - \text{fee})$

hold = $\max(\text{hold}, \text{cash} - \text{price})$

Part 8 — Counterexample Training

Before proving a greedy rule, spend 2–3 minutes trying to break it. A 30-second counterexample saves 20 minutes of wasted proof work.

Strategy	What it targets	Example
Very small inputs	Most greedy failures are visible with 3–5 elements.	Coin change [1,3,4] target 6 exposes failure with 3 coins.
Equal values	Ties often expose ambiguity in the greedy rule.	All equal activities: any order works → greedy trivially valid, not an attack.
One large outlier	A single large value often blocks better combinations.	Knapsack: one item weight=9, two items weight=5 each. Greedy picks large, misses 5+5.
Resource depletion	Design inputs where greedy uses a critical future resource.	Lemonade: give away all \$5 bills, then \$10 customer arrives → fail.
Worse now, better later	Directly tests future damage analysis.	Stock with fee: small daily gains eaten by fees; holding longer is better.
Nested overlapping intervals	Overlaps expose whether greedy respects structure.	Intervals: [1,100], [2,3], [4,5]. Greedy by start picks [1,100], misses 2 others.

Part 9 — Greedy Decision Tree

Use this during interviews and LeetCode. The condensed version at the bottom is what to memorize.

Full Decision Tree

START

↓

1. What is the repeated decision?

Cannot name one? → Not purely greedy

↓

2. What am I optimizing? (maximize / minimize / feasibility)

↓

3. What makes a choice invalid? (constraint)

↓

4. What resource runs out? (scarce resource)

↓

5. What does a bad choice destroy? (future damage)

↓

6. Propose a rule: "At every step, choose the ____"

↓

7. Try to break it (2 min, small/extreme/equal examples)

■
Broke it

↓
New rule

■
Did not break it

↓

8. Does subproblem repeat?

■

NO

↓

Suspicious

■

YES

↓

9. Future dependence?

■

YES

↓

Likely DP

■

NO

↓

10. Swap test

↓

Replace OPT's choice
with mine → still valid?

■

NO

↓

Try DP

■

YES

↓

11. State compress?

↓

1-2 variables?

■

NO

↓

DP smell

■

YES

↓

GREEDY ✓
→ Implement

One-Line Version

Condensed (memorize this):

CHOOSE → OPTIMIZE → CONSTRAINT → SCARCE → DAMAGE → RULE

→ ATTACK → SUBPROBLEM → FUTURE-DEPEND? → SWAP → STATE → IMPLEMENT

Part 10 — Problem-Solving Template

Copy this for every new greedy problem. Fill in the blanks before writing any code.

Problem-Solving Template

GREEDY PROBLEM TEMPLATE

PROBLEM: [One sentence]

CHOOSING: [Concrete repeated decision – no vague words]

OBJECTIVE: [Maximize / minimize / feasibility]

CONSTRAINT: [What makes a choice invalid?]

SCARCE: [What runs out / must be preserved?]

FUTURE DAMAGE: [Bad choice destroys what?]

RULE: "At every step, choose the _____ option."

REASON: [Why does this preserve future possibilities?]

COUNTEREXAMPLES:

1. Try: ____ Result: ____

2. Try: ____ Result: ____

3. Try: _____ Result: _____

SWAP TEST:

G = [my greedy choice]

X = [what OPT might choose instead]

Replace X with G $\rightarrow$ still valid because: _____

Objective unchanged because:

SUBPROBLEM: [Same structure after one choice? YES / NO]

STATE: [Variables to maintain: e.g. lastEnd, five, ten]

IMPLEMENTATION: [Sort / heap / two-ptr / hash]

COMPLEXITY: Time: $O(?)$ Space: $O(?)$

COMMON TRAPS: 1.

2.

Quick Reference — All 15 Problems

#	Problem	Greedy Rule	Pattern
1	Activity Selection	Earliest finish time	Preserve capacity

2	Longest Palindrome	Max even count + 1 center	Exchange
3	Assign Cookies	Smallest sufficient cookie	Smallest resource
4	Array Pair Sum	Pair adjacent sorted elements	Adjacent pairing
5	Lemonade Change	\$10 before \$5 for \$20	Flexible resource
6	DI String Match	I→smallest, D→largest	Flexible resource
7	Burst Balloons	Shoot at earliest end	Preserve capacity
8	Stock Buy/Sell II	Every upward consecutive step	Identity proof
9	Hand of Straights	Start at smallest card	Forced choice
10	Merge Triplets	Skip triplets exceeding target	Forced choice
11	Partition Labels	Split at furthest last occurrence	Exchange
12	Valid Palindrome II	Check both deletions at mismatch	Structural/forced
13	Valid Parenthesis *	Track [low,high] range	Range tracking
14	Largest Perimeter	Largest consecutive triple	Dominance
15	Candy	Two-pass left+right, take max	Exchange (two-sided)

This guide was built through working every problem from first principles — not memorization. The goal is to internalize the thinking process so you can apply it to any new problem you encounter.